Get Data

Feature C3 · Fetching Jira data — two equal paths, one artifact.

What is it?

The situational picture needs raw data from Jira. get_data offers two deliberately equal paths: the automated REST fetch — and the manual export, because in large organisations, approval for API access can take a long time. Both paths end in the same JSON file; the pipeline behind them is identical.

How to use it

```
set JIRA_TOKEN=YourAPIToken
python -m get_data fetch --url https://company.atlassian.net ^
  --project ART_A --email name@company.com --output ART_A.json
python -m get_data check ART_A_merged.json   # validate an export
```

Or via the GUI: python -m get_data (or the Get Data card in the launcher) — one window, both paths as a toggle: 'Jira REST fetch' and 'Existing export' with validation.

What it does

• API v3 (cursor) and v2 (offset) with full pagination, expand=changelog always set
• Auth for Cloud (e-mail + API token) or Server/DC (bearer PAT)
• Export validation catches the classics: missing changelog, forgotten follow-up pages, duplicates
• Error messages point 401/403 at the missing approval too — and at the manual fallback path

Guardrails

The token is never stored and never logged: the CLI reads it from an environment variable (JIRA_TOKEN), the GUI keeps it in memory only. Standard library only — no new dependencies. A contract test guarantees: a fetch and a manual export of the same data are processed identically.